Project Design Phase-II
Technology Stack (Architecture & Stack)

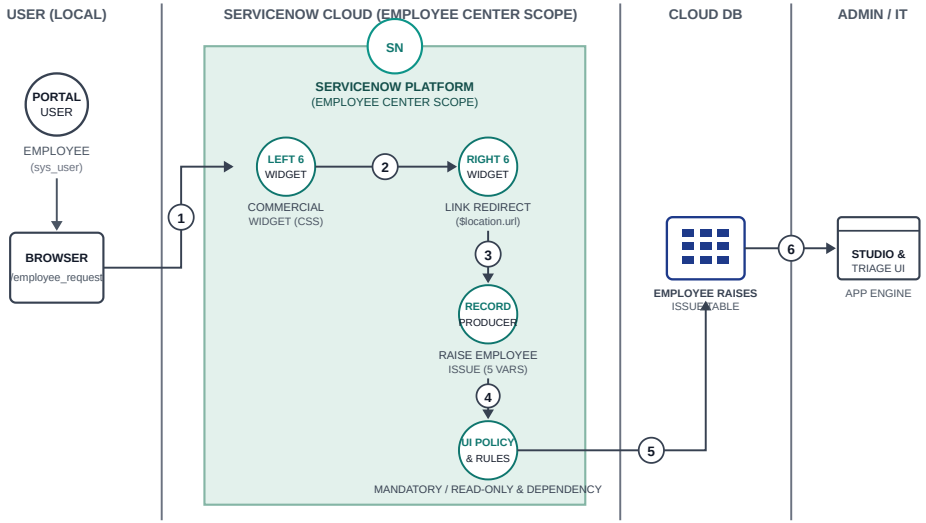
Date	24 October 2026
Team ID	Individual (Abdul Khadarvali)
Project Name	Employee Raise Issue – Record Producer & Service Portal Integration Guide
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Architecture Pattern: ServiceNow Scoped Application, Record Producer & Service Portal Integration (SkillWallet Specification)

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/



Guidelines:

- Include all the processes (As an application logic / Technology Block): Portal Widgets, Record Producer & UI Policies
- Provide infrastructural demarcation (Local / Cloud): Client Browser (Local) vs. ServiceNow Cloud PaaS (Employee Center)
- Indicate external / platform interfaces (API's etc.): AngularJS $location.url() & Service Catalog (sc_cat_item) API
- Indicate Data Storage components / services: Employee raises issue (Auto-numbered Table), sys_user & sys_user_group
- Indicate interface to rule / decision models (if applicable): Declarative UI Policies & Category → Subcategory Rules

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	How employees interact with the application: branded Service Portal (`/employee_request`) with a `6*6` responsive grid hosting `Commercial widget` and `Link redirect` widget.	HTML5, CSS3 (`linear-gradient`), JavaScript / AngularJS 1.x, ServiceNow Service Portal (`Employee Center Coral Theme`)
2.	Application Logic-1	Client-side widget redirection logic (`c.openRecordProducer()`) routing portal clicks to the `Raise Employee Issue` Record Producer and mapping 5 catalog variables (`Orders 100-500`).	AngularJS Client Script (`api.controller`, `$scope`, `$location.url`) & ServiceNow Record Producer

S.No	Component	Description	Technology
3.	Application Logic-2	Form validation logic enforcing `Short description` as Mandatory when `State is` New, and locking the `Number` field as permanently Read-only.	ServiceNow Table UI Policies & UI Policy Actions (`System UI → UI Policies`)
4.	Application Logic-3	Dynamic field dependency filtering `Subcategory` choices based on selected `Category` (`Network → VPN`, `Hardware → Laptop`, `Software → Server`, `Access → Forgot password`).	ServiceNow Dictionary Configuration (`Advanced view → Dependent Field = Category`)
5.	Database	Custom relational schema storing 8 issue fields: `Number` (Auto-Number), `Requester`, `Category`, `Subcategory`, `Short description`, `State`, `Priority`, and `Assignment Group`.	ServiceNow Glide Relational Database (MariaDB/MySQL-backed Custom Table: `Employee raises issue`)
6.	Cloud Database	Cloud-hosted platform master tables storing enterprise employee profiles and IT/HR support team groups linked via Reference fields.	ServiceNow Cloud Database (`sys_user` & `sys_user_group` system tables in `Employee Center` scope)
7.	File Storage	Storage for Record Producer catalog icons/pictures, portal theme stylesheet assets, and exported Application / Update Set XML packages.	ServiceNow Cloud Attachment & Image Store (`db_image` / `sys_attachment`) & Update Set XML
8.	External / Platform API-1	Service Portal Catalog Item endpoint used by the `Link redirect` widget to load the Record Producer form dynamically via `sys_id`.	ServiceNow Service Portal Catalog API (`sp?id=sc_cat_item&sys_id=<RECORD_PRODUCER_SYS_ID>`)
9.	External / Platform API-2	Reference lookup interface connecting Record Producer Variable Order 100 and table reference fields to live employee and group records.	ServiceNow Glide Reference Lookup API (`sys_user` & `sys_user_group`)
10.	Rule / Decision Model	Declarative condition evaluation engine replacing manual email triage with automated field enforcement and dependent choice filtering.	ServiceNow Condition Builder & Dictionary Choice Override Engine (No-Code / Low-Code)
11.	Infrastructure (Server / Cloud)	Application Deployment on Local System / Cloud: **Local Client Configuration:** Modern Web Browser (Chrome / Edge) accessing `/employee_request`. **Cloud Server Configuration:** ServiceNow Personal Developer Instance (PDI) running App Engine Studio.	ServiceNow Multi-Instance Cloud PaaS, ServiceNow Studio (Scoped App: `Employee Center`), Service Portal Designer

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Front-end single-page application (SPA) widget controller framework and responsive 12-column (6*6) grid layout used in Service Portal Designer.	AngularJS 1.x, Bootstrap 3 Grid System (6*6 Container), HTML5, CSS3
2.	Security Implementations	Scoped application namespace isolation, authenticated portal login page, read-only UI Policy protection on ticket `Number`, and role-based table access controls.	ServiceNow Scoped App Isolation (`Employee Center`), Portal Auth (`login`), UI Policy Read-Only Lock, HTTPS/TLS
3.	Scalable Architecture	3-tier decoupled architecture separating Presentation (`/employee_request` Portal & Widgets), Intake/Logic (`Raise Employee Issue` Record Producer & UI Policies), and Data (`Employee raises issue` table).	ServiceNow 3-Tier Cloud Architecture & App Engine Scoped Metadata Model

S.No	Characteristics	Description	Technology
4.	Availability	24/7 cloud-hosted availability across desktop and mobile browsers with dedicated portal routing for homepage (`ec_standard_home`), login (`login`), KB (`esc_knowledge_home`), and `404` error handling.	ServiceNow High-Availability Multi-Instance Cloud & Service Portal Server
5.	Performance	Sub-second widget rendering and client-side SPA transitions (`$location.url`) with declarative client-evaluated UI Policies and cached dictionary choices.	ServiceNow Client-Side SPA Caching, Declarative UI Policies & Indexed Reference Tables

References:

https://myskillwallet.ai/dashboard/skillwallet/module/servicenow-global-certification-lendi-6a38ca70d0c5068d7d216761/.../Employee-Raise-Issue-Record-Producer-Service-Portal-Integration-Guide

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d